

Extracting LLM Usage from Network Traffic

Zephaniah Roe
CMSC 25422

Abstract

In this project, I show that simple machine learning methods can be used to extract the number of prompt tokens, completion tokens, and the total cost from a packet trace of an LLM API call. For data collection I use OpenRouter to collect packet traces of API calls to GROK-3-MINI, GEMINI-2.5-FLASH-LITE, and LLAMA-3.2-3B-INSTRUCT. I evaluate on a test set from an entirely different packet capture from a different day. I find that a *single model* can predict from all three LLMs the number of completion tokens in an API call with an average absolute error of 248 and the number of prompt tokens with an average absolute error of 26.5. A single model can also predict the cost of an API call with high correlation to true values.

The complete code for this project can be found [here](#) and there are relevant sections of code in the appendix.

1 Motivation

Using code repositories with LLMs calls can be frustrating when there isn't clear outputs. First, users may wonder if their code is still running or how many API calls have been made. Second, they may worry about how much money they are spending because sometimes dashboards take a long time to update or users don't have admin access if they are using an employer's API key. Knowing what LLMs are running and how much money they are spending would be extremely useful so if there was a program running in the background that only relied on network data and not updating the code repository that could tell users this information, that could be extremely helpful to developers working with LLMs.

2 Data Collection

To collect data I run an automated script which makes a single API call at a time through OpenRouter. For each API call, the model is prompted with text from LMSYS-Chat-1M which contains real human chat data. After each API call, I make a request to a subdomain on my personal website which does not exist. This leaves an artifact in the packet trace looking like this:

```
Standard query response 0xf133  
No such name A 8928363d3c424f1b8b3320f91b90854c.zephaniahdev.com  
SOA ns1.dns-parking.com 58916 58916 53 53 7047
```

I can use this to separate the API calls with high reliability. In figure 1, the blue dots represent packets from OpenRouter and the vertical lines represent the timing of the DNS artifacts described

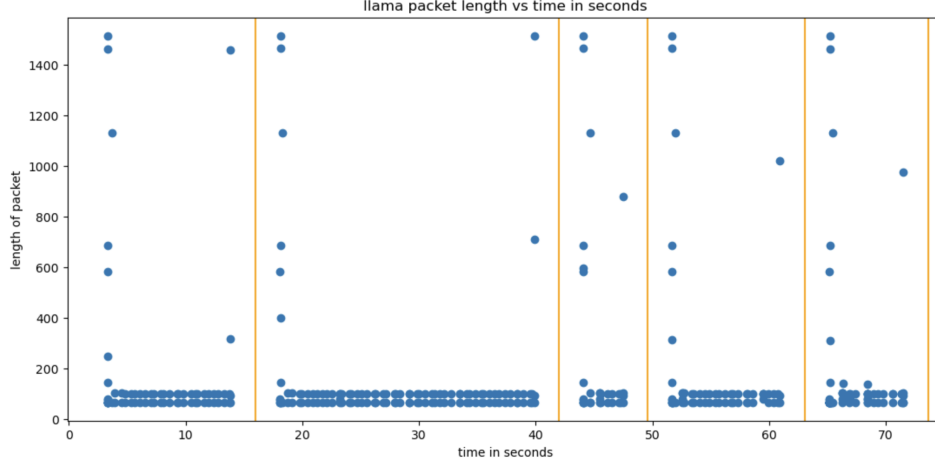


Figure 1: First 5 API calls in the LLAMA-3.2-3B-INSTRUCT packet trace. Vertical lines represent inserted DNS artifacts.

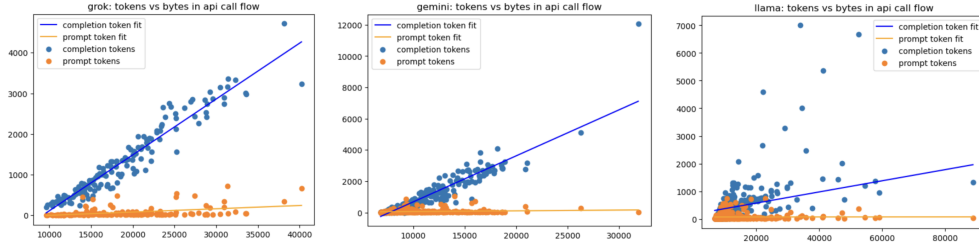


Figure 2: Completion and prompt tokens vs total number of bytes in API flow for GROK-3-MINI, GEMINI-2.5-FLASH-LITE, and LLAMA-3.2-3B-INSTRUCT.

above.

The function to generate data for a single API call can be found in the appendix section A.1 of this document. The rest of the relevant code can be found in the GitHub.

3 Motivating Issue: Where Linear Models Fail

By plotting the completion and prompt tokens vs the the total number of bytes in the API call flow we can see that there is a roughly linear relationship. If we train a linear model to predict the tokens based on only the number of bytes, we produce fairly resonable models. See figure 2 for results.

However, this only works for if we train a *different* regression for each of the LLMs. In reality, we may not have access to the name of the model being called. Therefore, it would be useful to fit a line to the entire dataset. In figure 3 we can see that this regression leaves a lot to be desired.

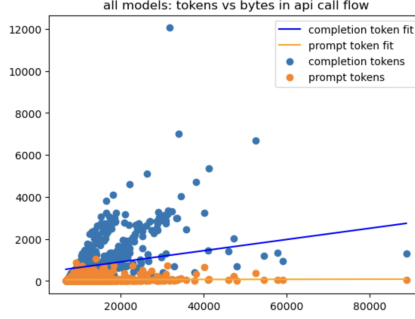


Figure 3: Completion and prompt tokens vs total number of bytes in API flow for GROK-3-MINI, GEMINI-2.5-FLASH-LITE, and LLAMA-3.2-3B-INSTRUCT for a single graph.

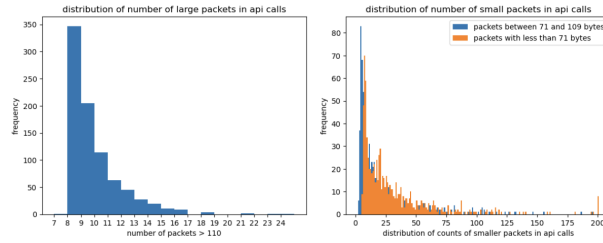


Figure 4: Distributions of large and small packets in the dataset. Note that if there are more than 200 packets in the figure to the left, I turn the count into 200 to account for the long tail of outliers. When building the data model in the section below I do the same.

4 Designing a Better Data Representation

In the previous section, we saw that training a regression on a single linear feature is not sufficient to produce a good model. Therefore, we try to design a better way to model each API call.

First, it is worth noting the much of the packet activity in each API call is highly repetitive. In figure 1 we can see that most packets contain very few bytes and happen in the time between an API is first called and when the final data is received. In the figure 4, we can see that distribution of the entire dataset. Most packets are very small and probably are just keeping the connection open. I believe that there is meaningful signal in the smaller packets, but because they are highly repetitive, so I chose not to represent them in high resolution. In the final data model I choose, I only represent the count of two ranges of small packets for each API call instead of modeling each individually.

For each API call I turn the trace into a length 43 tensor with the following values:

1. The number of packets between 71 and 109 bytes
2. The number of packets between 0 and 70 bytes
3. total length of larger packets (> 109 bytes) in the API call
4. total time for the API call
5. Time and length of the first 20 packets ≤ 109 bytes (time for the first packet is excluded because it is always 0)

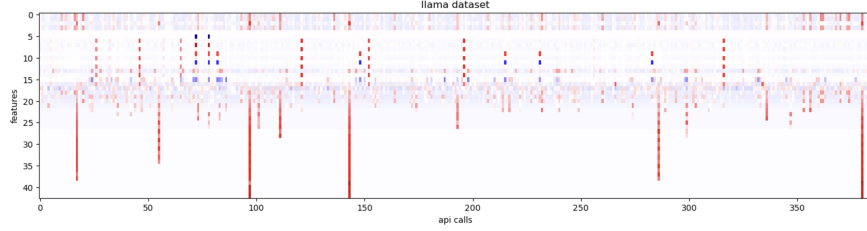


Figure 5: Tensor representing the LLAMA-3.2-3B-INSTRUCT training data.

The function of turning a packet trace into this format can be found in **Appendix section A.2**.

Figure 5 final tensor for the LLAMA-3.2-3B-INSTRUCT, we can see that the data is extremely diverse and there are very few API calls that max out the context length of 20 (have positive values up until the final feature).

5 Training the model

For cost, completion tokens and prompt tokens I train 5 models. One least squares, one ridge regression and an MLP with either 256, 128 or 32 values in its hidden layers. **The architecture of the MLP is in appendix section A.3.**

5.1 Training Loss for Completion Tokens

Below is the mean squared error and mean absolute error for each model trained to predict completion tokens in the LLMs. .

```
[vanilla_linear_completion] : mse = 2.64e+05, mae = 2.71e+02
[ridge_linear_completion] : mse = 2.66e+05, mae = 2.69e+02
[mlp_256_hidden_completion] : mse = 3.52e+03, mae = 34.0
[mlp_128_hidden_completion] : mse = 9.37e+03, mae = 55.4
[mlp_32_hidden_completion] : mse = 2.21e+04, mae = 86.9
```

5.2 Training Loss for Prompt Tokens

Below is the mean squared error and mean absolute error for each model trained to predict prompt tokens in the LLMs.

```
[vanilla_linear_prompt] : mse = 2.27e+03, mae = 22.5
[ridge_linear_prompt] : mse = 2.28e+03, mae = 22.6
[mlp_256_hidden_prompt] : mse = 36.5, mae = 3.52
[mlp_128_hidden_prompt] : mse = 21.7, mae = 3.22
[mlp_32_hidden_prompt] : mse = 1.35e+02, mae = 5.97
```

5.3 Training Loss for Predicting API Cost

Below is the mean squared error and mean absolute error for each model trained to predict cost of API calls in the LLMs.

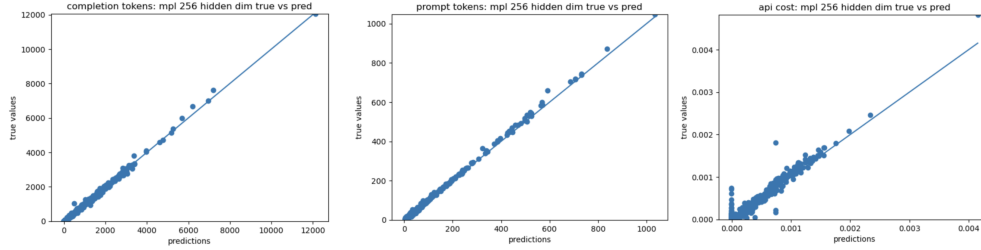


Figure 6:

```
[vanilla_linear_cost] : mse = 4.78e-08, mae = 0.000121
[ridge_linear_cost] : mse = 4.82e-08, mae = 0.000121
[mlp_256_hidden_cost] : mse = 1.82e-07, mae = 0.000267
[mlp_128_hidden_cost] : mse = 3.73e-08, mae = 9.86e-05
[mlp_32_hidden_cost] : mse = 9.16e-09, mae = 5.12e-05
```

Figure 6 shows that for each of these categories, the largest MLP is highly predictive on the train set of the true values.

5.4 Training Models

I use scikit-learn's

```
linear_model.LinearRegression()
```

to train the regular linear model and

```
linear_model.Ridge(alpha=.5)
```

to train the ridge regression. The MLP's trained have 3 layers with a the same hidden dimension at each hidden layer.

The full code for training the models can be found in appendix section A.4.

6 Testing

After all the training was done, I only then captured a packet trace for the test set. The test set is generated by calling the APIs with prompts that were not used to generate the train set. For each call a random model was selected.

For the same tasks as in section 4, we evaluate the trained models on this new test set:

6.1 Training Loss for Completion Tokens

```
[vanilla_linear_completion] : mse = 2.07e+05, mae = 3.29e+02
[ridge_linear_completion] : mse = 1.78e+05, mae = 3.03e+02
[mlp_256_hidden_completion] : mse = 2.31e+05, mae = 2.48e+02
[mlp_128_hidden_completion] : mse = 3.18e+05, mae = 2.57e+02
[mlp_32_hidden_completion] : mse = 2.52e+05, mae = 2.64e+02
```

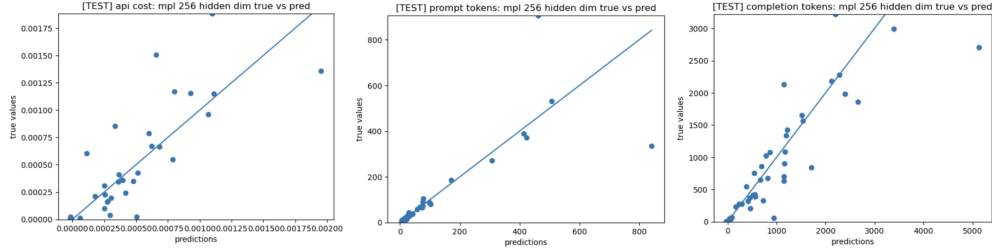


Figure 7: True vs predicted values in each setting on the test set.

6.2 Testing Loss for Prompt Tokens

```
[vanilla_linear_prompt] : mse = 2.19e+03, mae = 27.7
[ridge_linear_prompt] : mse = 3.35e+03, mae = 29.6
[mlp_256_hidden_prompt] : mse = 9.22e+03, mae = 26.5
[mlp_128_hidden_prompt] : mse = 1.75e+04, mae = 36.5
[mlp_32_hidden_prompt] : mse = 9.17e+03, mae = 26.4
```

6.3 Testing Loss for Predicting API Cost

```
[vanilla_linear_cost] : mse = 7.99e-08, mae = 0.000159
[ridge_linear_cost] : mse = 7.53e-08, mae = 0.000155
[mlp_256_hidden_cost] : mse = 2.46e-07, mae = 0.000328
[mlp_128_hidden_cost] : mse = 3.01e-08, mae = 0.000118
[mlp_32_hidden_cost] : mse = 5.86e-08, mae = 0.000135
```

6.4 Making sense of our results

The losses are much higher on the test set, but they are still highly predictive. The MLP does not significantly out perform the linear models in most cases. This tells us that a more complex model can lead to overfitting and the features we select are highly predictive with a linear model.

Figure 7 shows how even if we consider only the largest MLP, there is still a strong correlation in each of the evaluation categories.

7 Conclusion

Machine learning can be used to extract information about LLM calls in packet data. However, overfitting remains an issue with linear models being equally usefull as neural networks

A Appendix

A.1 Function for generating API activity:

```
1 def get_model_response(question):
2     # the code for calling the open router api was found in the open
   router website
3     # https://openrouter.ai/x-ai/grok-3-mini/api
```

```

4
5     client = OpenAI(
6         base_url="https://openrouter.ai/api/v1",
7         api_key=openrouter_key,
8     )
9
10    completion = client.chat.completions.create(
11        # model="x-ai/grok-3-mini",
12        # model="google/gemini-2.5-flash-lite",
13        model="meta-llama/llama-3.2-3b-instruct",
14        messages=[
15            {
16                "role": "user",
17                "content": question
18            }
19        ]
20    )
21
22    prompt_tokens = completion.usage.prompt_tokens
23    completion_tokens = completion.usage.completion_tokens
24    cost = completion.usage.cost
25
26
27    time.sleep(2)
28    try:
29        # this leaves a dns marker in the packet trace which helps
30        # differentiate
31        # what packets go with what request
32        # claude helped a little conceptually with this
33        # https://claude.ai/share/bd26116f-d738-4598-8fbd-c551845a6d1e
34        marker = f"{uuid.uuid4().hex}"
35        print("makingdnsrequest")
36        request = requests.get(f'https://{marker}.zephaniahdev.com/')
37    except:
38        pass
39    time.sleep(2)
40
41    return {
42        'prompt_tokens': prompt_tokens,
43        'completion_tokens': completion_tokens,
44        'cost': cost
45    }

```

A.2 Tuning Packet Trace Into Tensor

```

1 MAX_NUM_SMALL = 200
2 MAX_NUM_SMALLER = 200
3 MAX_PACKET_SIZE = 2_000
4 MAX_LARGER_BYTES_API_FLOW = 50_000
5 MAX_LARGER_PACKETS = 20
6 MAX_TIME = 60
7
8 def process_api_df_into_tensor(api_call):

```

```

9      rv = []
10
11      small_threshold = 110
12      smaller_threshold = 70
13
14      # num small packets
15      num_small = sum((api_call['Length'] < small_threshold) & (api_call['
16      Length'] > smaller_threshold))
17      rv.append(float(min(num_small, 200)))
18
19      # num smaller packets
20      num_smaller = sum(api_call['Length'] <= smaller_threshold)
21      rv.append(float(min(num_smaller, 200)))
22
23      # total_length of larger packets
24      total_len_larger = sum(api_call[api_call['Length'] >= small_threshold
25      ]['Length'])
26      rv.append(float(min(total_len_larger, MAX_LARGER_BYTES_API_FLOW)))
27
28      # total time
29      total_time = api_call.iloc[-1]['Time'] - api_call.iloc[0]['Time']
30      # print(total_time)
31      rv.append(float(min(total_time, MAX_TIME)))
32
33      larger_packet_data = [0] * (MAX_LARGER_PACKETS * 2 - 1)
34      min_time = float(min(api_call['Time']))
35
36      idx = 0
37      for _, row in api_call.iterrows():
38          if idx >= len(larger_packet_data):
39              break
40
41          if row["Length"] < small_threshold:
42              continue
43          try:
44
45              if row["Length"] == 0:
46                  print('here', row)
47
48              # time will always be zero for first packet so we don't need
49              # to record it
50              if idx == 0:
51                  larger_packet_data[0] = float(min(row["Length"],
52                  MAX_PACKET_SIZE))
53                  idx += 1
54              else:
55                  larger_packet_data[idx] = float(min(row["Length"],
56                  MAX_PACKET_SIZE))
57                  idx += 1
58                  larger_packet_data[idx] = float(min(row["Time"] - min_time
59                  , MAX_TIME))
60                  idx += 1
61          except:
62              print(idx)

```



```

57         assert False
58
59     return torch.tensor(rv + larger_packet_data)

```

A.3 MLP Architecture

```

1  class regression_mlp(nn.Module):
2  def __init__(self, hidden_dim=64):
3      super().__init__()
4      self.linear_relu_stack = nn.Sequential(
5          nn.Linear(43, hidden_dim),
6          nn.ReLU(),
7          nn.Linear(hidden_dim, hidden_dim),
8          nn.ReLU(),
9          nn.Linear(hidden_dim, 1)
10     )
11
12     def forward(self, x):
13         logits = self.linear_relu_stack(x)
14         return logits

```

A.4 Training Models

```

1  from sklearn.metrics import mean_absolute_error
2  from sklearn.metrics import mean_squared_error
3
4  def train_linear(train, labels):
5      reg = linear_model.LinearRegression()
6      reg.fit(train, labels)
7      train_predict = reg.predict(train)
8      return reg
9
10 def train_ridge(train, labels):
11     reg = linear_model.Ridge(alpha=.5)
12     reg.fit(train, labels)
13     train_predict = reg.predict(train)
14     return reg
15
16 def train_mlp(hidden_dim, datax, datay, num_batches=849 * 100, lr=1e-3):
17     model = regression_mlp(hidden_dim)
18     batch_size = 16
19     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
20
21     for b in range(num_batches):
22         batch = torch.zeros(16, 43)
23         batchy = torch.zeros(16,)
24
25         for i in range(batch_size):
26             random_idx = random.randint(0, len(datax) - 1)
27             y = datay[random_idx]
28

```

```

29         batch[i] = datax[random_idx]
30         batchy[i] = y
31
32         out = model(batch)
33         loss = sum((out.view(batch_size,) - batchy)**2) / len(batchy)
34
35         loss.backward()
36         optimizer.step()
37         model.zero_grad()
38
39         return model
40
41 # should take ~2 minutes to run
42
43 completion_models = dict()
44
45 completion_models['vanilla_linear_completion'] = train_linear(train_x,
46     completion_y)
47 completion_models['ridge_linear_completion'] = train_ridge(train_x,
48     completion_y)
49 completion_models['mlp_256_hidden_completion'] = trian_mlp(256, train_x,
50     completion_y)
51 completion_models['mlp_128_hidden_completion'] = trian_mlp(128, train_x,
52     completion_y)
53 completion_models['mlp_32_hidden_completion'] = trian_mlp(32, train_x,
54     completion_y)
55 # should take ~2 minutes to run
56
57 prompt_models = dict()
58
59 prompt_models['vanilla_linear_prompt'] = train_linear(train_x, prompt_y)
60 prompt_models['ridge_linear_prompt'] = train_ridge(train_x, prompt_y)
61 prompt_models['mlp_256_hidden_prompt'] = trian_mlp(256, train_x, prompt_y)
62 prompt_models['mlp_128_hidden_prompt'] = trian_mlp(128, train_x, prompt_y)
63 prompt_models['mlp_32_hidden_prompt'] = trian_mlp(32, train_x, prompt_y)
64 # should take ~2 minutes to run
65
66 cost_models = dict()
67
68 cost_models['vanilla_linear_cost'] = train_linear(train_x, costs_y)
69 cost_models['ridge_linear_cost'] = train_ridge(train_x, costs_y)
70 cost_models['mlp_256_hidden_cost'] = trian_mlp(256, train_x, costs_y)
71 cost_models['mlp_128_hidden_cost'] = trian_mlp(128, train_x, costs_y)
72 cost_models['mlp_32_hidden_cost'] = trian_mlp(32, train_x, costs_y)

```